

ML Models for Autonomous Machinery

Eres experto del MIT en Machine Learning (ML) y cuentas con más de 20 años de experiencia en este campo.

Quiero desarrollar un modelo de ML para maquinaria agrícola autónoma. El objetivo es tomar datos de los sensores (cámara, LiDAR, radar, IMU, GPS...) y establecer un recorrido óptimo para la maquinaria.

Por ejemplo, una cosechadora que tiene que realizar un trabajo en una finca, establecer ruta óptima, teniendo en cuenta obstáculos, dimensiones del vehículo, consumos, etc.

Otro ejemplo un tractor que tiene que mover material de un sitio a otro (ya definido) de manera autónoma. Como puede ser cargar o descargar un camión de pacas de paja.

Es posible que se necesiten varios modelos que funcionen de manera coordinada.

Te cuento lo que tiene que hacer el o los modelos:

- Determinar obstáculos que hay que evitar (árboles, animales, vehículos)
- Establecer recorridos optimizados
- Trabajar de manera precisa y adaptarse al medio, que puede ser variable
- Ser capaz de enganchar aperos a la maquinaria (ej. una cosechadora y su peine, un tractor con un remolque o arado...)

Dime:

Si con un modelo sirve, o se necesitan varios.

Los tres modelos de ML que mejor se adaptan para esto, y cuál/cuáles elegirías

Tres opciones de cómo crear el dataset o si puede servir alguno de Kaggle

Los 5 puntos más importantes que hay que tener en cuenta

Recuerda justificar y explicar de manera detallada todos los puntos

GPT-5.4

Conclusión corta

Para lo que planteas, **no recomiendo un único modelo end-to-end.**

Lo correcto en un sistema de maquinaria agrícola autónoma es una **arquitectura modular con varios modelos coordinados**:

1. **Percepción y fusión sensorial**
2. **Planificación de ruta y maniobra**
3. **Acople/manipulación fina de aperos**

Y alrededor de eso, bloques clásicos no-ML pero indispensables: **localización, SLAM, control, supervisión de seguridad y gestión de fallos.**

La búsqueda web va en esa línea: ya hay soluciones comerciales y prototipos que combinan **cámaras, LiDAR, GPS RTK, IMU y radar** para detectar obstáculos, mantener precisión centimétrica y reducir solapamientos/consumo, por ejemplo John Deere autónomo con 16 cámaras + LiDAR + visión 360º, automatización agrícola con GPS y autoguiado RTK con IMU, radar e ISOBUS.

1) ¿Sirve un solo modelo o hacen falta varios?

Mi respuesta: hacen falta varios

Un único modelo puede existir a nivel de investigación, pero **no es la mejor elección para un producto real**, y menos en maquinaria pesada. Las razones son:

- **Seguridad**: si un único modelo falla, falla todo el sistema.
- **Trazabilidad**: necesitas saber si el error fue de percepción, localización, planificación o control.
- **Mantenimiento**: será mucho más fácil mejorar un módulo concreto sin reentrenar todo.
- **Generalización**: el campo cambia muchísimo más que la carretera:
 - polvo
 - barro
 - vegetación alta

- animales
 - aperos diferentes
 - pendientes
 - GPS degradado bajo arbolado
-
- **Requisitos distintos:**
 - detectar un árbol o animal
 - calcular una ruta de cobertura óptima
 - hacer un acople fino a baja velocidad
- no son el mismo problema.

Arquitectura que sí recomiendo

A. Módulo 1: percepción y fusión

Entradas:

- cámara
- LiDAR
- radar
- IMU
- GPS/RTK

Salidas:

- obstáculos estáticos y dinámicos

- mapa de ocupación
- zonas transitables / no transitables
- estimación del terreno
- detección de personas, animales, vehículos, árboles, remolques, aperos

B. Módulo 2: localización y estado del vehículo

Idealmente con:

- GNSS RTK
- IMU
- odometría de ruedas
- LiDAR SLAM / visual SLAM
- calibración extrínseca muy cuidada

Aquí yo usaría **mucho clásico y poco ML**. En localización, normalmente la robustez de EKF/UKF/factor graphs + SLAM sigue siendo mejor apuesta que “todo con deep learning”.

C. Módulo 3: planificación

Dos niveles:

- **global**: cobertura de la finca, secuencia de pasadas, orden de tareas
- **local**: esquivar obstáculos, giros, maniobras, recálculo en tiempo real

D. Módulo 4: control y maniobra fina

- seguimiento de trayectoria
- control de velocidad

- maniobras marcha atrás
- aproximación a camión/remolque
- acople de aperos

E. Módulo 5: supervisor de seguridad

- parada segura
- watchdog
- detección de inconsistencia entre sensores
- fallback si se pierde RTK o visibilidad

Veredicto

Un modelo solo: posible en paper, mala idea en producto.

Varios modelos coordinados: sí, claramente.

2) Los 3 modelos de ML que mejor se adaptan

Aquí te doy las **3 familias de modelos** que mejor encajan en tu caso, con pros, contras y mi elección.

Modelo 1: Transformer multimodal en BEV para percepción y fusión

Tipo:

- BEVFusion
- BEVFormer
- variantes multimodales cámara + LiDAR + radar

Qué hace

Convierte la información de varios sensores en una representación común tipo **Bird's-Eye View**.

Eso te permite detectar:

- obstáculos
- espacio libre
- clases de objeto
- ocupación del entorno
- bordes de caminos o líneas de cultivo
- zonas peligrosas

Por qué encaja muy bien

Tu problema es esencialmente de **fusión sensorial espacial**.

Y la vista BEV es la más útil para maquinaria autónoma porque la planificación trabaja naturalmente sobre mapa/top-down.

Ventajas

- Muy bueno para combinar sensores heterogéneos
- Robusto ante oclusiones parciales
- Útil para 360° alrededor del vehículo
- Encaja bien con obstáculos y maniobras

Inconvenientes

- Mucha necesidad de datos sincronizados y bien calibrados
- Coste computacional alto

- Entrenamiento complejo
- Hay que vigilar latencia

Mi valoración

Es el modelo más importante de los tres.

Si tuviera que elegir solo uno para empezar, sería este.

Modelo 2: modelo de planificación aprendido para trayectorias

Tipo:

- imitation learning
- transformer temporal de trayectorias
- diffusion policy para motion planning
- trajectory prediction network

Qué hace

A partir de:

- mapa de ocupación
- estado del vehículo
- objetivo
- restricciones cinemáticas
- historial reciente

genera:

- trayectorias candidatas

- puntuación de maniobras
- acciones de corto horizonte

Por qué encaja

Tu caso no es solo “ir de A a B”, sino:

- minimizar solapamientos
- respetar anchura del apero
- evitar compactación innecesaria
- minimizar consumo
- girar en cabeceras
- adaptarse a obstáculos cambiantes

Un planificador aprendido puede capturar patrones que un planificador rígido no captura bien.

Ventajas

- Aprende maniobras “expertas”
- Puede adaptarse a escenarios complejos y variables
- Muy útil en cabeceras, maniobras y logística interna

Inconvenientes

- Fuera de distribución puede equivocarse
- No conviene dejarlo sin una capa de seguridad
- Necesita dataset de conducción/operación buena

Mi valoración

Lo elegiría, pero siempre con filtro de seguridad clásico:

- Hybrid A*
- lattice planner
- MPC
- collision checker

O sea: el ML propone; el sistema seguro valida.

Modelo 3: modelo de pose estimation / keypoints para acople de aperos

Tipo:

- keypoint detection
- instance segmentation + pose estimation
- 6D pose estimation
- visual servoing con red neuronal

Qué hace

Detecta y estima con precisión:

- posición del enganche
- geometría del apero
- orientación relativa tractor-apero
- distancia y ángulo en maniobra de aproximación

Por qué encaja

El acople de aperos **no es solo navegación**.

Es un problema de:

- percepción de muy corta distancia
- geometría fina
- control de precisión
- tolerancias pequeñas

Para “enganchar peine, remolque, arado, etc.” necesitas este bloque o algo equivalente.

Ventajas

- Muy útil en maniobras lentas y precisas
- Reduce dependencia exclusiva del operador
- Permite automatizar tareas muy valiosas

Inconvenientes

- Dataset muy específico
- Sensible a suciedad, polvo, iluminación y deformaciones
- No sustituye al control geométrico fino

Mi valoración

Lo incluiría sí o sí si el alcance real incluye aperos y carga/descarga.

Si el MVP solo hace navegación por finca, podría ir en fase 2.

Mi elección final

Si me pides qué elegiría yo:

Elegiría **los tres**, pero no todos con el mismo peso.

Para un MVP serio:

1. **Transformer multimodal BEV** para percepción
2. **Planificación híbrida:** ML + planner clásico + MPC
3. **Pose estimation para aperos**, pero en segunda fase si quieres acortar plazos

Si hay que priorizar por fases

- **Fase 1:** navegación autónoma en finca
- **Fase 2:** esquiva avanzada y optimización energética/logística
- **Fase 3:** acople de aperos y maniobras de carga/descarga

3) Tres opciones para crear el dataset

Opción 1: dataset propio real, instrumentando un tractor/cosechadora

Es la mejor opción para producto real

Qué recogería

- cámara delantera, trasera y laterales
- LiDAR 3D
- radar
- IMU

- GNSS RTK
- velocidad rueda/CAN bus
- ángulo de dirección
- estado del aparo
- consumo instantáneo
- patinamiento si es posible
- eventos del operador

Qué escenarios debes grabar

- día/noche
- polvo
- niebla ligera
- barro
- pendientes
- árboles
- animales
- otros vehículos
- personas
- parcelas con geometrías distintas
- cabeceras estrechas
- paso junto a remolques/camiones
- maniobras marcha atrás
- acople y desacople

Qué anotar

- 3D boxes / segmentación de obstáculos
- ocupación libre/no libre
- tipo de terreno
- trayectoria “experta”
- acciones del conductor
- puntos de acople
- episodios de fallo y casi-fallo

Por qué es la mejor

Porque tu problema tiene mucha especificidad:

- dimensiones reales del vehículo
- inercia
- aperos concretos
- suelo concreto
- geometría de tus fincas
- hábitos de operación reales

Eso no te lo da Kaggle.

Opción 2: dataset sintético con simulación / gemelo digital

Muy recomendable como acelerador

Sirve para generar:

- situaciones raras
- animales cruzando
- obstáculos inesperados
- GPS degradado
- maniobras de remolque
- acople repetido miles de veces
- cambios de iluminación y clima

Ventajas

- barato por muestra
- permite casos peligrosos sin riesgo real
- puedes obtener etiquetas perfectas
- ideal para entrenar política de planificación y docking

Riesgo principal

Domain gap: lo que aprende en simulación no siempre transfiere bien al campo real.

Cómo usarlo bien

- preentrenar en sintético
- afinar con real
- usar domain randomization
- mezclar texturas, barro, polvo, vegetación, aperos, cámaras sucias

Opción 3: dataset híbrido con públicos + tu dataset + Kaggle para subtareas

Esta es la opción más inteligente al principio

Puedes arrancar con datasets públicos para bootstrap y luego especializar con datos propios.

Recursos públicos útiles encontrados en la búsqueda

- Digital Agriculture Datasets: repositorio muy útil con datasets agrícolas y off-road; incluye navegación, segmentación, detección, 3D y herramientas para datos sintéticos.
- Awesome 3D LiDAR Datasets: buena base para localizar datasets de LiDAR, odometría y localización.
- EU Long-term Dataset with Multiple Sensors: muy interesante por incluir LiDAR, radar, cámaras, GNSS RTK e IMU con variación diaria y estacional.

¿Y Kaggle?

Sí, **puede servir**, pero **solo parcialmente**.

Kaggle sí sirve para

- preentrenar detector/segmentador visual
- aprender clases agronómicas
- cultivo vs mala hierba
- segmentación básica

Ejemplos vistos en los resultados:

- WeedCrop Image Dataset
- Plant Growth Segmentation Dataset

Kaggle no sirve, por sí solo, para

- navegación autónoma completa
- fusión de cámara + LiDAR + radar + IMU + GPS
- planificación de rutas reales
- control con cinemática de tractor/remolque
- acople de aperos

Mi recomendación de dataset

La mejor estrategia es:

1. **pretraining con públicos**
2. **simulación para rare cases**
3. **fine-tuning con dataset propio real**

4) Los 5 puntos más importantes a tener en cuenta

1. Seguridad funcional y arquitectura de fallback

Este es el punto más importante de todos.

Tu sistema no puede depender de “si la red acierta”.

Tiene que tener:

- parada segura
- geofence
- velocidad limitada por contexto
- monitor de confianza del modelo

- redundancia sensorial
- supervisor de colisión independiente
- estrategia ante pérdida de RTK, cámara cegada o LiDAR sucio

En maquinaria agrícola, un error no es un “bug visual”: puede romper una máquina, dañar cultivo, matar un animal o provocar un accidente grave.

2. Localización centimétrica, sincronización y calibración

Sin esto, el resto se derrumba.

Necesitas:

- GNSS RTK de calidad
- IMU robusta
- timestamps precisos
- calibración extrínseca cámara-LiDAR-radar-IMU
- compensación por inclinación/pendiente
- odometría auxiliar

La búsqueda lo confirma: el sector ya está usando RTK e IMU para precisión centimétrica y reducción de solapamientos/consumos, por ejemplo Kernova y Farmer Technology.

3. Definir bien la función objetivo de la ruta

“Ruta óptima” no significa “ruta más corta”.

En tu caso la función de coste debería incluir algo así:

$$J = w_1 \cdot t + w_2 \cdot E + w_3 \cdot R + w_4 \cdot M + w_5 \cdot C$$

donde:

t

= tiempo

E

= consumo energético/combustible

R

= riesgo / proximidad a obstáculos

M

= maniobras complejas o innecesarias

C

= compactación/impacto sobre suelo o cultivo

Y además meter restricciones duras:

- radio de giro
- anchura de máquina y apero
- gálibo
- estabilidad en pendiente
- velocidad máxima segura
- distancia mínima a obstáculos
- espacio de cabecera

Si no defines bien esto, puedes optimizar “matemáticamente” una ruta que operativamente sea mala.

4. Cobertura del dataset y edge cases

El modelo va a fallar donde no haya visto datos suficientes.

Tus edge cases reales serán:

- polvo levantado por la propia máquina
- ramas bajas
- animales quietos
- aperos parcialmente visibles
- remolques deformados o mal posicionados
- barro en sensores
- zonas de sombra dura
- GPS degradado bajo arbolado
- personas cerca durante maniobras de acople
- objetos no vistos en entrenamiento

Por eso el dataset debe tener:

- variedad estacional
- variedad de iluminación
- variedad de aperos
- variedad de fincas
- variedad de velocidades y cargas

En este sentido, son muy valiosos datasets con variación temporal y off-road como EU Long-term Dataset y los recopilados en Digital Agriculture Datasets.

5. El apero cambia la física, la visión y la planificación

Esto suele infravalorarse mucho.

No es lo mismo:

- tractor solo
- tractor + remolque
- cosechadora + peine
- tractor + arado
- tractor + pala cargando pacas

Cada configuración cambia:

- longitud total
- radio de giro
- barrido trasero
- punto de referencia de control
- masa e inercia
- visibilidad
- riesgo de colisión
- coste de maniobra

Por eso el sistema debe ser **configurable por implemento**.

No basta con “un modelo para todo” si el apero cambia radicalmente la dinámica y la geometría.

Recomendación final, muy directa

Si yo tuviera que diseñar esto con criterio industrial, haría esto:

Arquitectura recomendada

- **Percepción:** transformer multimodal en BEV
- **Localización:** RTK + IMU + odometría + SLAM clásico

- **Planificación global:** algoritmo clásico de cobertura + coste operativo
- **Planificación local:** modelo aprendido + filtro seguro
- **Control:** MPC / controlador geométrico
- **Aperos:** pose estimation específico para docking/manipulación

Estrategia de datos

- empezar con **datasets públicos**
- acelerar con **simulación**
- cerrar producto con **dataset propio real**

Prioridad de negocio/técnica

1. navegación autónoma básica en parcela
2. optimización de recorridos y obstáculos dinámicos
3. maniobras complejas de carga/descarga
4. acople automático de aperos

Mi respuesta final a tus preguntas, en una línea cada una

- **¿Un modelo o varios?**
Varios, claramente.
- **¿Los 3 modelos que mejor encajan?**
 1. Transformer multimodal BEV
 2. Planificador aprendido de trayectorias
 3. Pose estimation para aperos/acople

- **¿Cuál elegiría?**
Los tres coordinados, pero priorizando primero percepción + planificación.
- **¿Kaggle sirve?**
Sí, pero solo para subtareas visuales, no para la autonomía completa.
- **¿Qué es lo más importante?**
Seguridad, localización precisa, función objetivo correcta, dataset con edge cases y modelado específico del apero.

Si quieres, en el siguiente mensaje te puedo preparar una **arquitectura técnica completa extremo a extremo** para este proyecto con:

- diagrama de módulos
- stack software recomendado
- sensores mínimos viables
- roadmap por fases
- y una propuesta de dataset/etiquetado para un MVP realista.

Dame la arquitectura técnica a ir a un nivel muy técnico enfocado para el MVP realista:
diagrama de módulos
stack software recomendado
sensores mínimos viables
roadmap por fases
y una propuesta de dataset/etiquetado para un MVP realista.

GPT-5.4

Hecho: búsqueda web + síntesis técnica para un MVP realista

He realizado la búsqueda y, aunque muchos resultados son parciales o comerciales, sí aparece un patrón técnico bastante claro:

- En agricultura/autonomía ligera, los sistemas reales se apoyan muchísimo en **RTK-GNSS + IMU + percepción local** para mantener precisión y seguridad, como muestran soluciones de guiado y autosteer tipo AgriBus, Kernova y Topcon.
- Para autonomía robusta, la tendencia técnica es **fusión multi-sensor**; una revisión de 2025 concluye que combinar **LiDAR, cámara e IMU** mejora robustez frente a usar un solo sensor, y separa el sistema en **front-end** y **back-end** con fuerte peso en calibración, asociación y optimización Springer Review.
- En maquinaria/autonomía agrícola comercial, ya se ven arquitecturas con **GPS RTK + LiDAR + software de ruta + parada segura por fallo o riesgo**, como el caso GUSS/John Deere El Español.
- Para datasets y validación, hay valor en datasets multi-sensor sincronizados con **LiDAR, radar, cámara, GPS/IMU y ROS2 bags**, como CMHT, y recopilaciones amplias como Awesome-3D-LiDAR-Datasets.

Mi recomendación, por tanto, es un **MVP modular, no end-to-end**, con foco en una tarea concreta y repetible:

- **Caso MVP recomendado:** tractor/cosechadora en finca conocida, con geofencing, velocidad limitada, operación en rutas predefinidas o semipredefinidas, detección de obstáculos y replanning local.
- **No** empezaría el MVP por “enganchar aperos automáticamente”. Eso lo dejaría para fase 3.

1) Diagrama de módulos del MVP

Te dejo un diagrama técnico simplificado de una arquitectura realista:

```
<svg width="100%" viewBox="0 0 680 420" xmlns="http://www.w3.org/2000/svg" style="font-family:var(--font-sans);">
  <defs>
    <marker id="arrow" markerWidth="10" markerHeight="10" refX="8" refY="3" orient="auto"
markerUnits="strokeWidth">
      <path d="M0,0 L8,3 L0,6 Z" fill="var(--color-text-secondary)"></path>
    </marker>
  </defs>

  <rect x="18" y="16" width="644" height="388" rx="20" fill="var(--color-background-secondary)" stroke="var(--color-border-tertiary)" stroke-width="0.5"></rect>
```

```
<text x="36" y="40" fill="var(--color-text-primary)" font-size="16" font-weight="500">Arquitectura MVP realista · maquinaria agrícola autónoma</text>
<text x="36" y="58" fill="var(--color-text-secondary)" font-size="12">Finca conocida · RTK + IMU + percepción local · rutas seguras · parada segura</text>
```

```
<!-- Inputs -->
```

```
<rect x="36" y="92" width="120" height="44" rx="8" fill="var(--color-background-tertiary)" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="96" y="118" text-anchor="middle" fill="var(--color-text-primary)" font-size="13" font-weight="500">Cámaras RGB</text>
```

```
<rect x="36" y="146" width="120" height="44" rx="8" fill="var(--color-background-tertiary)" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="96" y="172" text-anchor="middle" fill="var(--color-text-primary)" font-size="13" font-weight="500">LiDAR 3D</text>
```

```
<rect x="36" y="200" width="120" height="44" rx="8" fill="var(--color-background-tertiary)" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="96" y="226" text-anchor="middle" fill="var(--color-text-primary)" font-size="13" font-weight="500">RTK-GNSS + IMU</text>
```

```
<rect x="36" y="254" width="120" height="44" rx="8" fill="var(--color-background-tertiary)" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="96" y="280" text-anchor="middle" fill="var(--color-text-primary)" font-size="13" font-weight="500">CAN / odometría</text>
```

```
<!-- Core pipeline -->
```

```
<rect x="202" y="82" width="172" height="62" rx="10" fill="#DDEBFF" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="288" y="106" text-anchor="middle" fill="#234" font-size="13" font-weight="500">Sincronización + calibración</text>
<text x="288" y="124" text-anchor="middle" fill="#456" font-size="11">timestamps · extrínsecos · health checks</text>
```

```
<rect x="202" y="162" width="172" height="68" rx="10" fill="#DDF5F1" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="288" y="188" text-anchor="middle" fill="#143" font-size="13" font-weight="500">Localización / SLAM</text>
<text x="288" y="206" text-anchor="middle" fill="#365" font-size="11">RTK + IMU + odometría + mapa local</text>
```

```
<rect x="202" y="248" width="172" height="68" rx="10" fill="#FFE8DC" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
<text x="288" y="274" text-anchor="middle" fill="#633" font-size="13" font-weight="500">Percepción / ocupación</text>
<text x="288" y="292" text-anchor="middle" fill="#754" font-size="11">obstáculos · espacio
```


libre · clases básicas</text>

```
<rect x="410" y="104" width="172" height="62" rx="10" fill="#EFE7FF" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
```

```
<text x="496" y="128" text-anchor="middle" fill="#324" font-size="13" font-weight="500">Planificador global</text>
```

```
<text x="496" y="146" text-anchor="middle" fill="#546" font-size="11">cabeceras ·  
cobertura · waypoints</text>
```

```
<rect x="410" y="186" width="172" height="62" rx="10" fill="#EFE7FF" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
```

```
<text x="496" y="210" text-anchor="middle" fill="#324" font-size="13" font-weight="500">Planificador local</text>
```

```
<text x="496" y="228" text-anchor="middle" fill="#546" font-size="11">evasión · recálculo  
corto horizonte</text>
```

```
<rect x="410" y="268" width="172" height="62" rx="10" fill="#EFE7FF" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
```

```
<text x="496" y="292" text-anchor="middle" fill="#324" font-size="13" font-weight="500">Control vehículo</text>
```

```
<text x="496" y="310" text-anchor="middle" fill="#546" font-size="11">steering · velocidad ·  
freno · PTO</text>
```

<!-- Safety -->

```
<rect x="590" y="96" width="54" height="246" rx="10" fill="#FFF1CC" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
```

```
<text x="617" y="136" text-anchor="middle" fill="#654" font-size="13" font-weight="500">Supervisor</text>
```

```
<text x="617" y="154" text-anchor="middle" fill="#654" font-size="13" font-weight="500">de</text>
```

```
<text x="617" y="172" text-anchor="middle" fill="#654" font-size="13" font-weight="500">seguridad</text>
```

```
<text x="617" y="210" text-anchor="middle" fill="#765" font-size="11">geofence</text>
```

```
<text x="617" y="226" text-anchor="middle" fill="#765" font-size="11">E-stop</text>
```

```
<text x="617" y="242" text-anchor="middle" fill="#765" font-size="11">sensor fail</text>
```

```
<text x="617" y="258" text-anchor="middle" fill="#765" font-size="11">safe stop</text>
```

<!-- Data loop -->

```
<rect x="202" y="342" width="380" height="42" rx="10" fill="var(--color-background-tertiary)" stroke="var(--color-border-secondary)" stroke-width="0.5"></rect>
```

```
<text x="392" y="367" text-anchor="middle" fill="var(--color-text-primary)" font-size="13" font-weight="500">Logging / Dataset / Replayer / Evaluación offline / Entrenamiento</text>
```

<!-- arrows -->

```
<path d="M156 114 L202 114" fill="none" stroke="var(--color-text-secondary)" stroke-width="1" marker-end="url(#arrow)"></path>
```

```

    <path d="M156 168 L202 168 L202 114" fill="none" stroke="var(--color-text-secondary)"
stroke-width="1" marker-end="url(#arrow)"></path>
    <path d="M156 222 L202 222 L202 114" fill="none" stroke="var(--color-text-secondary)"
stroke-width="1" marker-end="url(#arrow)"></path>
    <path d="M156 276 L202 276 L202 114" fill="none" stroke="var(--color-text-secondary)"
stroke-width="1" marker-end="url(#arrow)"></path>

    <path d="M288 144 L288 162" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
    <path d="M288 230 L288 248" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>

    <path d="M374 196 L410 196 L410 135" fill="none" stroke="var(--color-text-secondary)"
stroke-width="1" marker-end="url(#arrow)"></path>
    <path d="M374 282 L410 282 L410 217" fill="none" stroke="var(--color-text-secondary)"
stroke-width="1" marker-end="url(#arrow)"></path>
    <path d="M496 166 L496 186" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
    <path d="M496 248 L496 268" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>

    <path d="M582 135 L590 135" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
    <path d="M582 217 L590 217" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
    <path d="M582 299 L590 299" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>

    <path d="M288 316 L288 342" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
    <path d="M496 330 L496 342" fill="none" stroke="var(--color-text-secondary)" stroke-
width="1" marker-end="url(#arrow)"></path>
</svg>

```

2) Arquitectura técnica recomendada para el MVP

Objetivo del MVP

No intentar “autonomía general”.

Sí intentar:

- operar en **una finca conocida**

- con **trayectorias globales definidas o semidefinidas**
- a velocidad moderada
- con **detección de obstáculos**
- con **replanificación local**
- y con **parada segura** ante incertidumbre

Separación de módulos

A. Adquisición y sincronización

Responsabilidad:

- leer camera, lidar, imu, rtk, wheel odometry, can bus
- alinear timestamps
- monitorizar latencias
- verificar estado de sensor

Por qué es crítico:

La revisión de SLAM multi-sensor destaca que la complejidad práctica está en **preprocesado, calibración y acoplamiento entre sensores** [Springer Review](#).

B. Localización / SLAM

Responsabilidad:

- pose del vehículo en mapa
- fusión de RTK-GNSS + IMU + odometría
- mapa local cuando GNSS degrada

- corrección de drift

Justificación:

- En campo abierto, **RTK-GNSS + IMU** te da una base muy fuerte y barata para MVP AgriBus, Kernova.
- La literatura de fusión multi-sensor deja claro que **LiDAR/cámara/IMU** se compensan entre sí cuando uno falla o degrada Springer Review.

C. Percepción / ocupación

Responsabilidad:

- detección de obstáculos
- segmentación básica del espacio transitable
- occupancy grid / costmap
- seguimiento simple de objetos

Qué debe detectar el MVP:

- personas
- vehículos
- árboles/postes
- animales grandes
- maquinaria
- remolques/objetos voluminosos

No intentaría en MVP:

- clasificación fina de todas las especies
- razonamiento semántico complejo
- percepción 360° perfecta en cualquier clima

D. Planificación global

Responsabilidad:

- calcular orden de waypoints o pasadas
- minimizar solapes, maniobras innecesarias y kilómetros
- respetar geofence y zonas prohibidas

Aquí recomiendo **algoritmos clásicos**, no ML puro:

- cobertura por pasadas
- path planning sobre mapa de parcela
- penalización por giro/cabeceras

E. Planificación local

Responsabilidad:

- evasión inmediata
- replan corto horizonte
- desaceleración y safe stop

Aquí sí puedes introducir ML más adelante, pero para el MVP iría con:

- occupancy grid
- A* / Hybrid A* / lattice
- comprobación cinemática
- collision checking

F. Control

Responsabilidad:

- seguimiento de trayectoria
- control de velocidad
- control dirección
- frenado seguro
- gestión básica de PTO/implemento si aplica

G. Supervisor de seguridad

Responsabilidad:

- E-stop
- pérdida de RTK
- sensor ciego
- inconsistencia entre pose y percepción
- velocidad límite contextual
- parada segura

Esto no es opcional.

Casos reales de maquinaria autónoma comercial resaltan precisamente **detección de riesgo + parada automática** [El Español / GUSS](#).

3) Stack software recomendado

Voy a separarlo en **stack mínimo realista** y **stack ampliable**.

Stack mínimo realista para MVP

Sistema base

- Ubuntu LTS
- ROS 2 como middleware principal

Justificación:

- ROS2 aparece ya en datasets modernos multi-sensor como formato operativo útil para reproducción y validación CMHT.
- Te da:
 - pub/sub
 - bags
 - sincronización
 - nodos desacoplados
 - tooling para replays

Lenguajes

- C++ para tiempo real / percepción / control
- Python para entrenamiento, tooling y evaluación offline

Librerías núcleo

- OpenCV para cámara
- PCL para nubes de puntos
- Eigen para álgebra
- PyTorch para modelos ML
- optimización geométrica tipo Ceres o GTSAM para localización/fusión

Datos y experimentación

- ROS 2 bag como formato primario de captura
- Parquet/CSV para metadatos y eventos
- almacenamiento bruto en disco/objeto
- MLflow o equivalente para versionado de experimentos
- DVC o estructura similar para versionar datasets

Visualización y depuración

- RViz2
- panel propio simple para:
 - estado sensores
 - pose
 - trayectoria objetivo
 - coste local
 - alarmas

Infra de despliegue

- Docker para reproducibilidad
- CI básica para:
 - tests unitarios
 - replay tests sobre bags

- métricas de regresión

Stack ampliable en fase 2

- radar fusion
- mapa HD de finca
- inference optimizada en GPU
- planificador con learned cost
- teleoperación supervisada
- backend cloud para flota y analítica

Lo que no metería en el MVP

- arquitectura microservicios compleja
- multi-cloud
- MLOps pesado enterprise
- simulador hiperrealista antes de tener datos reales
- docking automático de aperos

4) Sensores mínimos viables

Aquí te doy una respuesta muy pragmática.

Configuración MVP recomendada

Opción A: la que yo elegiría

- **1 LiDAR 3D frontal** de buen FOV

- **2 cámaras RGB:**
 - delantera gran angular
 - trasera o lateral trasera según maniobra
- **RTK-GNSS**
- **IMU**
- **odometría / CAN bus / velocidad rueda**
- **E-stop físico**
- **encoder de dirección** si puedes leerlo
- **computadora edge con GPU**

Por qué esta combinación

- RTK-GNSS + IMU es la base de guiado centimétrico en agricultura de precisión AgriBus, Kernova, Topcon.
- LiDAR aporta geometría local, muy útil cuando:
 - hay polvo parcial
 - cambia la iluminación
 - necesitas medir obstáculos con precisión
- El propio material técnico general sobre LiDAR resalta que sistemas precisos usan **LiDAR + IMU + GPS** IBM.
- En aplicaciones agrícolas autónomas reales se ve **RTK + LiDAR** como núcleo El Español / GUSS.

¿Meter radar en MVP?

Mi recomendación: **no al principio**, salvo que:

- vayas a operar con polvo frecuente muy denso
- niebla
- baja visibilidad recurrente
- o quieras robustez extra contra objetos móviles

El radar ayuda, pero incrementa mucho:

- calibración
- sincronización
- complejidad de fusión
- coste de etiquetado

¿Cámara sola?

Para un MVP muy barato de autoguiado entre líneas, quizá.

Para tu caso de autonomía con obstáculos reales, **no la recomiendo sola**.

¿Sensores mínimos absolutos si quieres recortar costes?

- RTK-GNSS
- IMU
- una cámara frontal
- CAN/odometría

Pero eso sería más bien un **autoguiado asistido**, no una autonomía robusta.

5) Roadmap por fases

Fase 0 — Instrumentación y captura

Duración orientativa: 4–8 semanas

Objetivo:

- montar sensores
- sincronizar
- grabar bags
- calibrar intrínsecos/extrínsecos
- tener tooling de replay

Entregables:

- pipeline de captura
- calibración reproducible
- viewer con overlay de pose y nube

Criterio de salida:

- grabaciones repetibles en varias jornadas
- deriva de pose bajo control
- dataset inicial usable

Fase 1 — Autoguiado seguro en finca conocida

Duración: 6–10 semanas

Capacidades:

- geofence
- waypoints
- control de velocidad

- seguimiento de línea/ruta
- parada segura ante fallo sensor/RTK

Sin aún:

- evasión compleja de obstáculos
- aperos automáticos
- operaciones logísticas complicadas

Éxito:

- completar recorridos simples de forma consistente

Fase 2 — Percepción y evasión local

Duración: 8–12 semanas

Capacidades:

- occupancy grid local
- detección de obstáculos
- recálculo local
- bypass o safe stop

Éxito:

- evitar obstáculos estáticos simples
- detenerse ante dinámicos o inciertos

Fase 3 — Optimización operativa

Duración: 8–12 semanas

Capacidades:

- mejor cobertura de parcela
- menos giros improductivos
- menos solapamiento
- coste por consumo/tiempo

Aquí ya puedes introducir:

- learned cost maps
- ranking de trayectorias
- ajuste por apero

Fase 4 — Maniobras complejas / docking de aperos

Duración: 10–16 semanas

Capacidades:

- marcha atrás precisa
- alineación con remolque/camión
- acople con keypoints/pose estimation

Mi consejo:

No la metas en el MVP inicial.

6) Propuesta de dataset y etiquetado para un MVP realista

Estrategia general

No intentes etiquetar “todo”.

Etiqueta solo lo que mueve las decisiones del MVP.

Tipos de datos a capturar

Para cada sesión:

- camera frames
- lidar point clouds
- rtk pose
- imu
- wheel speed / steering / can
- comandos de control
- estado del apero
- eventos manuales
- estado supervisor
- condiciones: hora, clima, polvo, parcela, cultivo, implemento

Escenarios que sí debes grabar

- caminos abiertos
- cabeceras
- bordes de parcela
- árboles aislados
- postes/vallas
- otros vehículos
- animales grandes si aparecen
- personas
- diferentes velocidades
- distintas iluminaciones

- polvo moderado
- GNSS parcialmente degradado

Escenarios que no priorizaría al inicio

- barro extremo
- niebla severa
- noche cerrada
- acople automático
- varios vehículos cooperando

Niveles de etiquetado

Nivel 1 — imprescindible

Para arrancar MVP:

- pose ground truth
- trayectoria ejecutada
- eventos de intervención humana
- zonas transitables / no transitables
- obstáculos estáticos con caja o máscara simple
- obstáculos dinámicos con timestamp y clase básica

Clases iniciales:

- person
- vehicle

- tree/post
- animal_large
- implement/trailer
- unknown_obstacle

Nivel 2 — recomendable

- occupancy labels
- líneas de cultivo o bordes navegables si aplican
- tipo de superficie:
 - tierra
 - vegetación
 - camino
- zonas de riesgo

Nivel 3 — más adelante

- tracking multiobjeto
- intenciones
- severidad de riesgo
- calidad de visibilidad
- incertidumbre de sensor

Formato de dataset recomendado

Datos brutos

- ROS2 bags por sesión

Justificación: datasets modernos multi-sensor ya publican bags sincronizados CMHT.

Índice y metadatos

- session_id
- field_id
- vehicle_config
- implement_config
- weather
- time_of_day
- operator
- notes

Etiquetas

- JSON/Parquet por frame o por segmento
- separar:
 - percepción
 - trayectoria
 - eventos
 - safety

Propuesta concreta de tamaño inicial

Para un MVP serio pero aún manejable:

Dataset v0

- 30–50 horas de operación real
- 10–15 parcelas o configuraciones
- 3–5 condiciones de luz
- 3–4 configuraciones de implemento como máximo
- 200–500 eventos de obstáculo relevantes
- 50–100 casos de intervención humana bien documentados

Split

- 70% train
- 15% val
- 15% test
- separando por **día/parcela**, no por frames aleatorios

Esto es clave. Si mezclas frames del mismo recorrido en train y test, te engañas.

7) Qué datasets públicos sí usaría

No hay un Kaggle mágico que te resuelva esto.

Pero sí usaría públicos para **bootstrap**.

Para estructura multi-sensor / tooling

- CMHT: útil por tener **LiDAR + radar + IR + cámara + GPS/IMU** sincronizados y bags.
- Awesome-3D-LiDAR-Datasets: útil para localizar datasets de odometría/localización/segmentación y comparar sensores.

Para entender sensor suite real

- La tesis de Universidad Autónoma Chapingo menciona explícitamente arquitectura con **encoder/odometría, GPS, IMU y LiDAR con SLAM**, que es muy alineada con un MVP terrestre.

Importante

Úsalos para:

- pretraining
- tooling
- evaluación de pipelines
- validar arquitectura

No para:

- asumir que generalizan a tu finca sin fine-tuning

8) Mi recomendación final, muy directa

Si yo diseñara tu MVP, haría esto:

MVP 1

- operación en finca conocida
- RTK-GNSS + IMU + LiDAR + 2 cámaras + CAN
- rutas globales predefinidas
- percepción local básica
- evasión simple o safe stop

- velocidad limitada
- supervisor de seguridad duro

MVP 2

- mejora de ocupación semántica
- optimización de recorridos
- más robustez frente a polvo / GNSS degradado
- telemetría y analítica de flota

MVP 3

- docking / aperos
- remolque
- logística con carga/descarga

9) Resumen ejecutivo

Diagrama de módulos

Sí: modular, con

- sensores
- sincronización/calibración
- localización/SLAM
- percepción
- planificación global
- planificación local

- control
- supervisor de seguridad
- logging/training

Stack recomendado

- Ubuntu LTS
- ROS 2
- C++ + Python
- OpenCV + PCL + PyTorch
- bags para captura/replay
- Docker + CI básica

Sensores mínimos viables

Mi elección:

- RTK-GNSS
- IMU
- LiDAR 3D frontal
- 2 cámaras RGB
- CAN/odometría
- E-stop

Roadmap

- F0 instrumentación

- F1 autoguiado seguro
- F2 percepción + evasión
- F3 optimización
- F4 apéros/docking

Dataset/etiquetado

- 30–50 h iniciales
- bags sincronizados
- labels de transitabilidad, obstáculos, trayectoria, intervención y safety
- clases básicas, no taxonomía compleja

Si quieres, en el siguiente mensaje te preparo una versión todavía más práctica:

1. **BOM de hardware mínimo** con rangos de precio
 2. **estructura exacta de carpetas del dataset**
 3. **esquema de topics ROS2** para este MVP
 4. **métricas de validación** tipo success rate, intervention/km, false stop rate, path efficiency
-